

INT DAO · SECURITY REVIEW

# INT DAO

## BoostManager PickCallbackV2 NftDistributorV2.

A security review of BoostManager PickCallbackV2 NftDistributorV2.

---

| CLIENT  | PROJECT                                            | AUDITORS | AUDIT WINDOW                         | FINAL REPORT    |
|---------|----------------------------------------------------|----------|--------------------------------------|-----------------|
| INT DAO | BoostManager<br>PickCallbackV2<br>NftDistributorV2 | Guardian | August 26 –<br>September 27,<br>2024 | October 4, 2024 |

§ 00

# Contents.

34 pages — overview, confidence ranking, scope, 22 findings, disclaimer, and about Guardian.

|      |                                    |       |
|------|------------------------------------|-------|
| § 01 | <b>Overview</b>                    | P. 03 |
| § 02 | <b>Confidence Ranking</b>          | P. 04 |
| § 03 | <b>Scope &amp; Classifications</b> | P. 05 |
|      | Files in Scope                     | P. 06 |
| § 04 | <b>Findings</b>                    | P. 07 |
|      | Main Review · 18 findings          | P. 09 |
|      | Remediation Review · 4 findings    | P. 29 |
| § 05 | <b>Disclaimer</b>                  | P. 33 |
| § 06 | <b>About Guardian</b>              | P. 34 |

# Overview.

Guardian reviewed BoostManager PickCallbackV2 NftDistributorV2 across 2 review rounds. The final report includes 3 High, 9 Medium, 10 Low findings.

AUDIT FIRM

Guardian

CLIENT

INT DAO

PROJECT

BoostManager PickCallbackV2 NftDistributorV2

PREPARED BY

Guardian

AUDIT WINDOW

August 26 – September 27, 2024

FINAL REPORT

October 4, 2024

REPOSITORY

BowTiedPickle/int-contracts

METHODOLOGY

Smart contract · Manual · Main Review ·  
Remediation Review

MAIN REVIEW COMMIT

d17db9c40502

## 1.1 Confidence Ranking

3 / 5

Moderate  
Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

| 1                      | 2                 | 3                      | 4                  | 5                       |
|------------------------|-------------------|------------------------|--------------------|-------------------------|
| VERY LOW<br>CONFIDENCE | LOW<br>CONFIDENCE | MODERATE<br>CONFIDENCE | HIGH<br>CONFIDENCE | VERY HIGH<br>CONFIDENCE |

## 1.2 Findings at a Glance

| SEVERITY                     | TOTAL | PENDING | ACK | PARTIAL | RESOLVED |
|------------------------------|-------|---------|-----|---------|----------|
| <span>■</span> Critical      | 0     | 0       | 0   | 0       | 0        |
| <span>■</span> High          | 3     | 0       | 1   | 0       | 2        |
| <span>■</span> Medium        | 9     | 0       | 4   | 1       | 4        |
| <span>■</span> Low           | 10    | 0       | 5   | 0       | 5        |
| <span>■</span> Informational | 0     | 0       | 0   | 0       | 0        |
| All severities               | 22    | 0       | 10  | 1       | 11       |

## § 02

# Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

| CONFIDENCE RANKING                              | DEFINITION AND RECOMMENDATION                                                                                                                                                                                               | RISK PROFILE                                                                              |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>5 Very High Confidence</b>                   | Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit. | 0 High/Critical findings and few Low/Medium severity findings.                            |
| <b>4 High Confidence</b>                        | Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.    | 0-1 High/Critical findings per engagement week and little to no Medium severity findings. |
| <b>3 Moderate Confidence</b><br><b>ASSIGNED</b> | Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.                                                            | Some High/Critical findings, with various Medium severity findings.                       |
| <b>2 Low Confidence</b>                         | Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.                                                                     | Over 6 High and Critical findings across the engagement.                                  |
| <b>1 Very Low Confidence</b>                    | The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.                                                                           | Over 12 High and Critical findings across the engagement.                                 |

## § 03

# Scope & Classifications.

Guardian reviewed BoostManager PickCallbackV2 NftDistributorV2 in `BowTiedPickle/int-contracts`.

## 3.1 Severity Matrix

|            | IMP · HIGH | IMP · MED | IMP · LOW |
|------------|------------|-----------|-----------|
| LIK · HIGH | Critical   | High      | Medium    |
| LIK · MED  | High       | Medium    | Low       |
| LIK · LOW  | Medium     | Low       | Low       |

## IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

## IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

## IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

## LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

## LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

## LIKELIHOOD · LOW

Unlikely to ever occur in production.

# Files in Scope.

| FILE                             | NSLOC | TOTAL |
|----------------------------------|-------|-------|
| No files were recorded in scope. |       |       |

## § 04

# Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

| ID          | TITLE                                           | CATEGORY      | SEVERITY | STATUS             | PAGE  |
|-------------|-------------------------------------------------|---------------|----------|--------------------|-------|
| MAIN REVIEW |                                                 |               |          |                    |       |
| H-01        | Time Boost Rewards Are Not Accurate             | Logical Error | HIGH     | ACKNOWLEDGED       | p. 9  |
| H-02        | Gaining Rewards That Should Have Been Forfeited | Logical Error | HIGH     | RESOLVED           | p. 11 |
| H-03        | Existing NFTs Cannot Be Synced When Disabled    | Logical Error | HIGH     | RESOLVED           | p. 12 |
| M-01        | Auto Processing Not Working As Expected         | Logical Error | MEDIUM   | ACKNOWLEDGED       | p. 13 |
| M-02        | Staking While Boost Disabled                    | Logical Error | MEDIUM   | RESOLVED           | p. 14 |
| M-03        | Improper Decimal Math                           | Rounding      | MEDIUM   | RESOLVED           | p. 15 |
| M-04        | Reward Gaming In Epochs                         | Gaming        | MEDIUM   | RESOLVED           | p. 16 |
| M-05        | Staking Epoch 0                                 | Gaming        | MEDIUM   | RESOLVED           | p. 17 |
| M-06        | Blacklist Causes Auto-Process DoS               | Logical Error | MEDIUM   | PARTIALLY RESOLVED | p. 18 |
| L-01        | Minimum Distribution Count May Not Be Reached   | Documentation | LOW      | ACKNOWLEDGED       | p. 19 |
| L-02        | Distributor Balance Transfer Is Not Possible    | Warning       | LOW      | RESOLVED           | p. 21 |
| L-03        | addPoolRewardToken Should Have Duplicate Check  | Validation    | LOW      | RESOLVED           | p. 22 |
| L-04        | Burning When Boost Disabled Causes Reward Loss  | Logical Error | LOW      | ACKNOWLEDGED       | p. 23 |
| L-05        | Lack of _startTimestamp Validation              | Validation    | LOW      | RESOLVED           | p. 24 |

## Findings continued.

| ID   | TITLE                                     | CATEGORY       | SEVERITY | STATUS       | PAGE  |
|------|-------------------------------------------|----------------|----------|--------------|-------|
| L-06 | Epoch Start And End Time Inconsistent     | Documentation  | LOW      | ACKNOWLEDGED | p. 25 |
| L-07 | Unused Receiver Config Value              | Best Practices | LOW      | RESOLVED     | p. 26 |
| L-08 | Auto-Processing Index Is Reset            | Griefing       | LOW      | RESOLVED     | p. 27 |
| L-09 | Token Transfers May Cause Loss Of Rewards | Logical Error  | LOW      | ACKNOWLEDGED | p. 28 |

### REMEDIATION REVIEW

|      |                                               |               |        |              |       |
|------|-----------------------------------------------|---------------|--------|--------------|-------|
| M-01 | Lost Rewards Due To A Single Blacklisted User | Logical Error | MEDIUM | ACKNOWLEDGED | p. 29 |
| M-02 | Epoch 0 Time Staking Works                    | Validation    | MEDIUM | ACKNOWLEDGED | p. 30 |
| M-03 | Users Total Earned Decreases                  | Logical Error | MEDIUM | ACKNOWLEDGED | p. 31 |
| L-01 | Multi-Burns and Mints Exceed Block Gas Limit  | Gas Usage     | LOW    | ACKNOWLEDGED | p. 32 |

HIGH **H-01**

ACKNOWLEDGED

# Time Boost Rewards Are Not Accurate

|          |                           |
|----------|---------------------------|
| CATEGORY | Logical Error             |
| LOCATION | BoostManager.sol: 350-356 |
| REVIEW   | Main Review               |

## — DESCRIPTION

Users earn different kinds of rewards based on boost types, and the time boost is one of them. According to the documentation and governance proposal, the time boost is based on the minting date of active NFTs and should be linear. Users who hold NFTs longer should receive higher time boost rewards.

"The older your IFT is the larger the reward compared to others. For example somebody who minted/won an IFT 2 months ago will have twice their time boost reward compared to someone who minted 1 month ago."

The protocol uses a stake balance of 1 for each epoch to calculate the time boost. A user's time boost balance will be 1 for holding the NFT for one epoch. The boost balance will be 2 for holding it for 2 epochs, and so on. These balances are updated in the `_updateTokenId` function.

However, the time boost balance is not accurately reflected in the earned rewards. Currently, a user who stakes an NFT and waits 4 epochs without taking any action will receive a time boost based on a balance of 1 for all 4 epochs, after which their balance will be updated to 4. The earned rewards would be calculated as  $1 * 4$  (balance \* rpt). In contrast, another user who calls the `process` function in every epoch would receive a reward of  $(1 * 1) + (2 * 1) + (3 * 1) + (4 * 1)$ , since this user's balance would correctly reflect the time-based boost.

Utilizing both the time difference and the increased staked balance of the user causes rewards to become exponential rather than linear, creating an opportunity for malicious users to game the system.

## — RECOMMENDATION

Consider a laddering approach to apply the appropriate multiplier for each epoch the user was staked in.

PROOF OF CONCEPT <https://github.com/GuardianAudits/int-1/pull/1/files>

---

## H-01 Time Boost Rewards Are Not Accurate

CONTINUED

---

### — RESOLUTION

Status: Acknowledged.

HIGH **H-02**

RESOLVED

# Gaining Rewards That Should Have Been Forfeited

CATEGORY Logical Error  
LOCATION NftDistributorV2.sol  
REVIEW Main Review

## — DESCRIPTION

Unstaking is not performed when a user burns their NFT while the boost is disabled. The NFT gets burnt, but the rewards for that token ID remain unchanged until the sync function is called. The `sync` function resolves these state inconsistencies and forfeits rewards from that token ID.

However, the `sync` is not automatically invoked, and there is a time gap between burning an NFT and calling `sync`. Additionally, any user can mint a new NFT with the exact same token ID after it has been burnt.

When a user burns their NFT while the boost is disabled, another user can mint a new NFT with the same ID before the `sync` is called, leading them to gain the previous user's rewards that should have been forfeited.

## — RECOMMENDATION

Consider allowing a user to claim their tokenId rewards even when boost is disabled and/or invoke the `sync` function directly from the `_processBurn` when a user burns their NFT while the boost is disabled.

PROOF OF CONCEPT <https://github.com/GuardianAudits/int-2/pull/1>

## — RESOLUTION

Status: Resolved.

HIGH **H-03**

RESOLVED

# Existing NFTs Cannot Be Synced When Disabled

CATEGORY Logical Error  
LOCATION BoostManager.sol: 401  
REVIEW Main Review

## DESCRIPTION

The BoostManager contains function `sync` so that mint and burn actions performed when the boost was disabled can be reflected within the BoostManager.

For example, when a user has an existing stake and burns their NFT when boost is disabled, the stake balance will remain until `sync` is called. However, the current handling logic for `isDisabled` leaves a token's stake balance open to manipulation when `boostDisabled()` is toggled.

Consider the following scenario:

1. Boost is disabled and Bob has an existing stake.
2. Bob burns NFT; BASE balance is still greater than 0.
3. Boost is enabled; Because `else` branch is now entered and the NFT does not exist, function `sync` continues and does not actually sync.
4. Alice mints that same tokenId and it is staked. The stake balance is now double the `TOKENS_PER_NFT`.

Ultimately, Alice was able to double her BASE balance since Bob's was never removed from the system.

## RECOMMENDATION

Either ensure all tokenIds are synced prior to re-enabling boost, or have sync be called automatically.

PROOF OF CONCEPT <https://github.com/GuardianAudits/int-2/pull/2>

## RESOLUTION

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

# Auto Processing Not Working As Expected

CATEGORY Logical Error  
LOCATION BoostManager.sol: 423-429  
REVIEW Main Review

## — DESCRIPTION

The protocol has an auto processing for token Ids, and this is done in a linear order with a tracking index. Processing starts from the latest `autoProcessedIndex` and continues.

However, `_tokenIds` is an `EnumerableSet` and it doesn't have a certain order. Anytime a token is burnt or unstaked, the last element of the set is moved to the burnt token's index and the last element is popped.

As a result of this, the unprocessed last token in the set might be moved to an already processed index, causing it to remain unprocessed for a longer time and decrease the compounding effect of the rewards for that token.

## — RECOMMENDATION

Consider specifically calling the `process` for the token that is being moved from the last index to the new index during burns/unstakes if the new index is smaller than `autoProcessedIndex`. Otherwise, consider using an ordered data structure.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

# Staking While Boost Disabled

CATEGORY Logical Error  
LOCATION BoostManager::L305  
REVIEW Main Review

## DESCRIPTION

`_process()` does not validate if the protocol has set `boostDisabled` to `true`. This will allow users to compound their staked assets even when the boost is meant to be disabled. This allows already staked users to gain an edge over newly staked users when the boost is reenabled.

## RECOMMENDATION

`_process()` should verify that `boostDisabled` is `false` and otherwise return early.

## RESOLUTION

Status: Resolved.

MEDIUM **M-03**

RESOLVED

# Improper Decimal Math

CATEGORY Rounding  
LOCATION BoostManager::L207  
REVIEW Main Review

## DESCRIPTION

The reward calculation is correct when using tokens that have 18 decimal places. However if a token that uses six decimals is used, such as USDC, a rounding error can occur.

Consider the following scenario:

Distribution Tokens : 500e6

Epoch Length : 604800

Remaining Rewards : 0

Reward Rate : 826.72

TokensPerNft : 100,00e18

NFTs Minted : 100

Total Supply : 10,000,000e18

Last Interaction : 2

RPT : 1.65e-4

With Solidity, `rpt` will be rounded down to 0, and no tokens will be distributed.

## RECOMMENDATION

To properly calculate `rpt`, you must consider the token decimals of the reward token being used. Consider normalizing the decimals for the `rpt` calculation and then converting back to the proper decimals for transfers.

PROOF OF CONCEPT [https://github.com/GuardianAudits/int-1/tree/POC\\_ROUNDING\\_ERROR](https://github.com/GuardianAudits/int-1/tree/POC_ROUNDING_ERROR)

## RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

## Reward Gaming In Epochs

CATEGORY Gaming  
LOCATION BoostManager::L305  
REVIEW Main Review

### — DESCRIPTION

`process()` will get all the rewards for all boost types, then stake them to `BASE`. This will increase a user's balance, and, in turn, allow them to receive more reward tokens throughout the boost. Then, `process()` will call `_claim()`, which will send the same tokens that were just staked to the user (or their vault).

A user can extract more rewards by calling `process()` repeatedly throughout their staking duration. This will allow them to out-earn users who deposit at the same time as them by actively engaging with `process()` and compounding their rewards.

The change in distribution is at the expense of other staked users, as there is a set amount of tokens to be distributed and the final rewards are skewed towards the user due to their continuous compounding.

### — RECOMMENDATION

Consider processing existing tokenIds in a scheduled manner to avoid gaming. Otherwise, clearly document this to users. Restricting users to process at only certain times will add too much complexity.

### — RESOLUTION

Status: Resolved.

MEDIUM **M-05**

RESOLVED

# Staking Epoch 0

CATEGORY Gaming  
LOCATION Global  
REVIEW Main Review

## DESCRIPTION

`boostDisabled` is initialized as `false`, which allows users to stake in epoch zero. If a user stakes in epoch zero, it will set the `cachedLastUpdateTime` to a value before `startTimestamp`.

Then when epoch one starts, the `rpt` calculation will assign a higher value than expected based off the abnormal `cachedLastUpdateTime` value. This will lead to a user (or users) to earn more rewards than are allocated for the epoch.

## RECOMMENDATION

Disable the boost in the constructor and do not enable it until the first epoch has started.

PROOF OF CONCEPT <https://github.com/GuardianAudits/int-2/pull/3>

## RESOLUTION

Status: Resolved.

MEDIUM **M-06**

PARTIALLY RESOLVED

## Blacklist Causes Auto-Process DoS

CATEGORY Logical Error  
LOCATION BoostManager.sol  
REVIEW Main Review

### — DESCRIPTION

The function `process` uses a push pattern to send user rewards. Because the pool config does not restrict which tokens can be used as reward tokens, if even one reward token has blacklist or freeze functionality, it can potentially brick auto-processing and reward token transfers.

### — RECOMMENDATION

Consider utilizing a pull-over-push pattern.

### — RESOLUTION

Status: Partially resolved.

LOW **L-01**

ACKNOWLEDGED

# Minimum Distribution Count May Not Be Reached

CATEGORY Documentation  
LOCATION NftDistributorV2.sol: 117-123  
REVIEW Main Review

## — DESCRIPTION

According to the protocol [docs](https://internet-token.gitbook.io/internet-token/how-does-it-work#ift-distribution), a minimum of 10 NFTs should be distributed every week. Since the protocol has been live for a while, both `distributionCountPerSet` and `minDistributionCountPerSet` are currently set to 10.

The NFT distribution is based on the total ticket amount for the week and `distributionSpacing`. However, the weekly distribution count might fall below 10, even if more than 10 tickets are sold.

For example:

Ticket sold: 11

`_distributionCountPerSet` : 10

`distributionSpacing` will be 2.

In this case, winner ticket ids based on the `vrf` result will either be:

- 0, 2, 4, 6, 8, 10 or
- 1, 3, 5, 7, 9

From the perspective of a user who invests in the protocol based on the documentation, having 1 of 11 tickets should provide a greater than 90% chance of earning an NFT (10/11). However, in both scenarios mentioned above, the number of distributed NFTs is significantly below the minimum weekly amount, reducing the user's chance of getting an NFT to around 50%.

## — RECOMMENDATION

Document this behavior on the website protocol documentation and let users know that there might be some cases where the minimum number of NFTs cannot be distributed.

---

**L-01** Minimum Distribution Count May Not Be Reached

CONTINUED

---

— **RESOLUTION**

Status: Acknowledged.

LOW **L-02**

RESOLVED

## Distributor Balance Transfer Is Not Possible

CATEGORY Warning  
LOCATION Global  
REVIEW Main Review

### — DESCRIPTION

The `NftDistributor` contract holds a portion of the reward tokens and transfers 100,000 tokens to the corresponding `NftVault` whenever a lottery winner receives an NFT.

However, the currently active `NftDistributor` contract lacks a transfer function to move all of its reward token balances to the new distributor contract.

To resolve this, the reward token admin can burn the entire balance of the old distributor and mint an equivalent amount to the new distributor. However, the `StakingAirdropDeployment` contract, which is responsible for setting up the new distributor, does not perform this action during deployment. This means there will be a gap between deploying the new contract and minting reward tokens to it, which could cause some distributions to fail depending on the timing of the deployment.

### — RECOMMENDATION

Consider implementing the burning and minting of reward tokens within the `deploy` function to ensure a seamless migration. Alternatively, ensure that the deployment timing is precise, so there won't be a lottery distribution between deploying the contract and minting the balance to it.

### — RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

# addPoolRewardToken Should Have Duplicate Check

CATEGORY Validation  
LOCATION BoostManager.sol: 530  
REVIEW Main Review

## DESCRIPTION

`addPoolRewardToken` function does not have a check to prevent adding same token address twice. This function adds a new entry to `poolConfigs` but there isn't a way to remove a config from it. Adding the same config twice would cause incorrect state updates during regular action flows.

There's an assumption that a pushed pool config with a non-INT\_POOL\_ID also does not have INT as a reward token. If such a pool config was indeed added, then the token callbacks won't be properly disabled when adding the reward to the BoostManager with function `addRewards`:

```
if (poolId == INT_POOL_ID) _rewardToken.disableCallbacks();
```

## RECOMMENDATION

Consider adding a duplicate entry check for this function.

## RESOLUTION

Status: Resolved.

LOW **L-04**

ACKNOWLEDGED

## Burning When Boost Disabled Causes Reward Loss

CATEGORY Logical Error  
LOCATION NftDistributorV2.sol: 296-308  
REVIEW Main Review

### — DESCRIPTION

Users have the ability to burn their NFTs through the distributor contract. If the boost feature is enabled, users' stakes will be force unstaked and any earned rewards will be processed before the INT tokens are transferred back to the user.

In the event that a user chooses to burn their NFT while the boost feature is disabled, they will lose any unprocessed rewards that were accumulated while the boost was active.

### — RECOMMENDATION

Document this behavior for users and inform them that there is a possibility of losing rewards if they burn their tokens while the boost feature is disabled. Also consider suggesting to users to manually process their rewards before burning their tokens.

### — RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

## Lack of `_startTimestamp` Validation

CATEGORY Validation  
LOCATION StakingAirdropDeployment::L35  
REVIEW Main Review

### — DESCRIPTION

In the constructor of `StakingAirdropDeployment.sol`, `_startTimestamp` is only validated that it must be larger than the current timestamp. However, in `BoostManager.sol` the start timestamp must be at least one day in the future. If the `_startTimestamp` is set to less than one day it will prevent deployment of `BoostManager.sol` and require you to redeploy `StakingAirdropDeployment.sol`.

Additionally, a 30 day window is allowed in `BoostManager.sol`, but only a 1 week window is allowed in `StakingAirdropDeployment.sol`.

### — RECOMMENDATION

Validate that the `_startTimestamp` is greater than one day from time of deployment.

### — RESOLUTION

Status: Resolved.

LOW **L-06**

ACKNOWLEDGED

## Epoch Start And End Time Inconsistent

CATEGORY Documentation  
LOCATION BoostManager.sol: 241, 245  
REVIEW Main Review

### — DESCRIPTION

If a user calls `getEpochStartTimestamp(0)`, the result is 0.

If a user calls `getEpochEndTimestamp(0)`, the result is `startTimestamp - 1`.

This delta between start and end timestamp is not the length of an epoch `PERIOD` and may confuse external systems.

### — RECOMMENDATION

Clearly document this behavior.

### — RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

## Unused Receiver Config Value

CATEGORY Best Practices  
LOCATION BoostManager.sol  
REVIEW Main Review

### — DESCRIPTION

In the BoostManager contract the `receiverConfig` value is written and stored on the `poolConfigs`, however is never used after assignment.

### — RECOMMENDATION

Implement the use-case for the `receiverConfig` or consider removing it.

### — RESOLUTION

Status: Resolved.

LOW **L-08**

RESOLVED

## Auto-Processing Index Is Reset

CATEGORY Griefing  
LOCATION BoostManager.sol:387  
REVIEW Main Review

### — DESCRIPTION

The current `_processUnstake` logic resets the `autoProcessedIndex` to 0 if an unstake causes the index to be out-of-bounds with the new tokenId set.

A user could burn their NFT just before a user stakes, so that the `autoProcessedIndex` is reset, and there may have to be multiple `process()` calls before the newly staked tokenId is reached.

### — RECOMMENDATION

Consider documenting this potential behavior.

### — RESOLUTION

Status: Resolved.

LOW **L-09**

ACKNOWLEDGED

# Token Transfers May Cause Loss Of Rewards

CATEGORY Logical Error  
LOCATION BoostManager.sol: 283-291  
REVIEW Main Review

## — DESCRIPTION

NFT holders earn both INT and WETH rewards. INT rewards are re-staked and held in the `NftVault` contract, while WETH rewards are directly transferred to the NFT owner. The owner of the `NftVault` is the same person who holds the NFT. This means that the ownership of the `NftVault` will change whenever the NFT is transferred to someone else.

When users transfer their NFTs, they will lose all of their unprocessed reward. They would also lose all INT rewards even if they process it.

An inactive holder might transfer or sell their NFT at a lower price without knowing that the NFT also carries additional internal value in the form of rewards. Since this behavior is not documented, the holder might mistakenly think they receive their rewards passively each week.

## — RECOMMENDATION

Consider not only re-staking INT rewards but also allowing users to claim their INT rewards to their own wallet in a manner similar to WETH rewards. Additionally, document this behavior and advise users to `process` their rewards before selling their NFTs on the secondary market.

Another option to consider is calling the `process` function in a `_beforeTokenTransfer` callback anytime an NFT transfer happens.

## — RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

## Lost Rewards Due To A Single Blacklisted User

CATEGORY Logical Error  
LOCATION BoostManager.sol  
REVIEW Remediation Review

### — DESCRIPTION

The `disablePool` function is introduced to address issue M-06. When a pool is paused using this function, it is excluded from reward processing. However, there is currently no way to unpause a pool afterward.

If even a single user is blacklisted, the auto-processing will be DoS'ed. To resolve this, admins will disable the pool. However, this will cause all other users who are not blacklisted to miss out on their rewards, as the pool will no longer distribute rewards.

### — RECOMMENDATION

Clearly document this behavior to users, or introduce some exception logic to allow non-blacklisted users to claim their rewards.

This can mean excluding specific addresses rather than pausing the entire pool. Additionally, consider adding a function to enable pools afterward.

### — RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

# Epoch 0 Time Staking Works

CATEGORY Validation  
LOCATION BoostManager::L311  
REVIEW Remediation Review

## DESCRIPTION

`BoostManager.sol` can be deployed so that the first epoch can happen up to 30 days after the deployment. A user can stake in epoch 0 and grow their balance of time based rewards since the calculation in `_processTime()` is based off the amount of time a user has been staked divided by an epoch's length. This gives users who deposit in epoch 0 an unfair advantage over other users

## RECOMMENDATION

Consider disabling staking in epoch 0 or updating the `joinedTimestamp` to the start time of the first epoch.

PROOF OF CONCEPT [https://github.com/GuardianAudits/int-1/blob/POC\\_Epoch0\\_Time/test/guardian/pocs/Epoch0.sol](https://github.com/GuardianAudits/int-1/blob/POC_Epoch0_Time/test/guardian/pocs/Epoch0.sol)

## RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

# Users Total Earned Decreases

CATEGORY Logical Error  
LOCATION BoostManager.sol  
REVIEW Remediation Review

## — DESCRIPTION

Users are now able to process their TIME boost balance prior to updating the tokenId. This will increase the total supply of this boost, which would ultimately lower the reward per token.

Consider the following scenario:

1. Bob sends a transaction to process and claim to receive X earnings returned by function `totalEarned`
2. Alice who is in cooldown processes right before Bob and updates their time balance, increasing TIME total supply and decreasing the reward per token.
3. Bob's transaction goes through afterwards and receives  $<X$  tokens, which is less earnings than initially reported.

Note that frontrunning is not necessary, and will happen naturally as multiple tokenId's are iterated through with `process()`. Ultimately, Bob's total earnings decreased although staking earnings should typically monotonically increase. This is especially unexpected since `totalEarned` should accurately reflect how much is claimable to users.

## — RECOMMENDATION

Consider processing the TIME boost after updating token metadata. Ensure users do not lose unclaimed rewards.

PROOF OF CONCEPT <https://github.com/GuardianAudits/int-fuzzing/blob/3fb46506d6d254186590a76412010ca4655ead61/test/invariant/helpers/ForgeTester.sol#L30>

## — RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

## Multi-Burns and Mints Exceed Block Gas Limit

CATEGORY Gas Usage  
LOCATION Global  
REVIEW Remediation Review

### — DESCRIPTION

The tests `test_gas_multiBurn`, `test_gas_multiForceBurn`, and `test_gas_multiMint` now fail and show usage of more than 120,000,000 gas.

### — RECOMMENDATION

Adjust the failing tests.

### — RESOLUTION

Status: Acknowledged.

§ 05

# Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

## About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

**\$45B+**

DIGITAL ASSETS UNDER  
REVIEW

**200+**

ENGAGEMENTS COMPLETED

**500+**

CRITICAL & HIGH FINDINGS  
REPORTED

Reach us at [audits@guardianaudits.com](mailto:audits@guardianaudits.com).